Mock PS Submission: Quadrotor Single Motor Failure

Task 1b:

Reference Literature: M. W. Mueller and R. D'Andrea, "Stability and control of a quadrocopter despite the complete loss of one, two, or three propellers," IEEE International Conference on Robotics and Automation (ICRA), 2014.

The Quadrotor is fully defined by 12 parameters -

- The position vector.
- The velocity vector.
- The quaternion for its orientation.
- The angular velocities in all the 3 principal directions. Taken as → $\omega_b = (p, q, r)$

We have 4 parameters in our control which are the thrust forces (which are proportional to the square of the angular velocities of each of the fans) and we want to translate these to 4 active parameters of the quadrotor which influence the equations of motions for the quadrotor. Which are-

- T - The collective along the body's Z direction
- τ_x - The roll torque in the body's X direction
- τ_y - The pitch torque in the body's Y direction
- τ_z - The yaw torque in body's Z direction

Considering a drone kept in the X-Y plane with the propellers at (0,+-l,0) and (+-l,0,0) with the adjacent propeller's blades rotating in opposite directions.

The lift generated by each of the propellers is - $k_f \omega_p^2$ where ω_p is the angular velocity of each rotor.

The total lift → T = $f_1 + f_2 + f_3 + f_4$

Roll torque → $\tau_x = (f_2 - f_4)\, l$

Pitch torque → $\tau_y = (f_3 - f_1)\, l$

Yaw torque → $\tau_z = \tau_1 + \tau_2 + \tau_3 + \tau_4$

Where τ_n is the torque of the independent propeller which equates to - $\tau_n = (-1)^n f_i k_\tau$

(since adjacent rotors rotate the opposite way to cancel out the net torque)

Therefore the yaw torque → $\tau_z = -k_\tau(-f_1 + f_2 - f_3 + f_4)$

The transformation matrix looks like -

$$\begin{bmatrix} T \\ \tau_x \\ \tau_y \\ \tau_z \end{bmatrix} = \begin{bmatrix} 1 & 1 & 1 & 1 \\ 0 & l & 0 & -l \\ -l & 0 & l & 0 \\ k_\tau & -k_\tau & k_\tau & -k_\tau \end{bmatrix} \begin{bmatrix} f_1 \\ f_2 \\ f_3 \\ f_4 \end{bmatrix}$$

The net torque is the one which we get from this control matrix plus the drag torque which opposes the body rotation which is given by $\tau d = (0, 0, -\gamma r)$ where r is the z component of the body's angular velocity. (The components in the other two directions are taken to be zero from simplicity as the quadrotor would mostly have negligible angular velocities in the other two directions.) So the net torques on the body of the drone are-

$\tau_x = (f_2 - f_4)l$

$\tau_y = (f_3 - f_1)l$

$\tau_z = k_\tau(f_1 - f_2 + f_3 - f_4)$ - γr

These torques can be related to the angular accelerations and velocities of the quadcopter using -

$I_b\omega_b' + \sum_{i=1}^{4} I_p\omega'_{pi} + \omega_b \times H = \tau_{res}$ equation - 1

where,

I_b is the inertia tensor of the drone body

I_p is the inertia tensor for each of the propeller blades

$\omega_b \; and \; \omega_p$ is the angular momentum of the body and the blades respectively

H is the total angular momentum of the entire system which is given as

$H = I_b\omega_b + \sum_{i=1}^{4} I_p(\omega_{pi} + \omega_b)$

- For simplification ignoring the $\sum_{i=1}^{4} I_p\omega'_{pi}$ term in equation 1 because ω_p s of the rotors is gonna be almost constant.
- Also ignoring the blade's angular momentum in the expression for H ($\sum_{i=0}^{4} I_p(\omega_{pi} + \omega_b)$) because $I_b >> I_p$.

So we are left with this equation after all the assumptions and simplifications -

$I_b\omega_b' + \omega_b \times [I_b\omega_b] = \tau_{res}$

Using the simplification that $I_{xx} \; and \; I_{yy}$ are equal because of the drone's symmetry, solving this matrix equation using the normal cross product methods we get the final expressions for the angular accelerations for the drone-

$$p' = \frac{(f_2 - f_4)l - (I_{zz} - I_{yy})qr}{I_{xx}} \quad q' = \frac{(f_3 - f_1)l - (I_{xx} - I_{zz})pr}{I_{yy}} \quad r' = \frac{k_\tau(f_1 - f_2 + f_3 - f_4) - \gamma r}{I_{zz}}$$

And the linear expression for the vertical equilibrium is →

$md'' = \sum_{i=0}^{4} f_i - mg$

There are forces in the other 2 directions too due to the component of this lift force due to the tilt of the drone and will have similar equations but won't have gravity and the lift force will just be a component in the respective direction.

This was all in ideal case with all the rotors working properly.

Now when one of the rotors is dead -

$$\begin{bmatrix} T \\ \tau_x \\ \tau_y \\ \tau_z \end{bmatrix} = \begin{bmatrix} 1 & 1 & 1 \\ l & 0 & -l \\ 0 & l & 0 \\ -k_\tau & k_\tau & -k_\tau \end{bmatrix} \begin{bmatrix} f_2 \\ f_3 \\ f_4 \end{bmatrix}$$

Now this is a problem because we have 4 parameters in the left hand side to be controlled via 3 independent values. This is not possible and we will lose one degree of freedom. We have a choice to decide which PID controller to give up between the 4 drone parameters to give up.

- We cannot give up lift because then the drone would just fall while keeping the orientation the same.
- If we try to give up τ_y and try to maintain τ_z as zero, we see the resulting expression for τ_y as → $\tau_y = f_3 l$. A net positive torque which makes the drone flip and crash into the ground. This is not acceptable and we must take $\tau_y = 0$.
- For that we have to limit f_3, but then that would mean that the expression for τ_z is $\tau_z = -k_\tau(f_2 + f_4)$ which is the only acceptable case as this only causes the drone to spin on its Z axis and doesn't change the thrust axis and this is the basis for the relaxed hover idea from Mueller and D'Andrea's paper.

Calculating the steady state yaw angular velocity -

Assuming the tilt of the z axis of the drone to be θ, for vertical equilibrium, $(f_2 + f_4)cos(\theta) = mg$ (because f_3 was set to near zero), therefore

$$f_2 = f_4 = \frac{mg}{2 \cdot cos(\theta)}$$

Now from the resultant net yaw torque at steady state-

$\tau_z = \tau_{motor} + \tau_{drag} = k_\tau(f_2 + f_4) - \gamma r = 0$

Therefore,

$$r = \frac{k_\tau(f_2+f_4)}{\gamma} = \frac{k_\tau \cdot mg}{cos(\theta)\gamma}$$

Fault Detection

We will go over the Residual observer method for fault detection as it's very lightweight and fast as compared to say something like a neural network.
It basically calculates the expected angular acceleration and the actual angular acceleration read by the IMU (Inertial measurement unit) and takes the decision as to whether the motor is fine or not depending on if the error is higher than a specified threshold.
From this equation -

$I_b \omega_b' + \omega_b \times [I_b \omega_b] = \tau_{res}$

We get

$\omega_{b\,expected}' = I_b^{-1}[\tau_{res} - \omega_b \times [I_b \omega_b]]$, where τ_{res} is the expected resultant torque with all motors fine.

Using the IMU the real angular acceleration is measured using -

$$\omega_{b\,measured}' = \frac{\omega_k - \omega_{k-1}}{\Delta t}$$

And then the residual error r(t) is measured using -

$$\omega_{b\ measured}'(t) - \omega_{b\ expected}'(t) = r(t) = \begin{bmatrix} r_x(t) \\ r_y(t) \\ r_z(t) \end{bmatrix}$$

Say the threshold acceleration is - $\delta(t)$, we get the motor failure by -

- If $r_x(t) > \delta(t)$ motor 4 failed
- If $r_x(t) <- \delta(t)$ motor 2 failed
- If $r_y(t) > \delta(t)$ motor 1 failed
- If $r_y(t) <- \delta(t)$ motor 3 failed

Task 1d:

For the simulation I've chosen to do in C for the speed and the familiarity with the language.
We will be using an headerfile raymath.h which is an open-source component of the raylib family which gives us the functions and structs for basic vector mathematics and quaternion usage.
For the orientation or the attitude, we chose to use quaternions over euler angles to avoid gimbal locks which can interfere here as the drone might flip quite a bit when using unideal corrections.
The file structure is a simple 3 file system with the header (failure.h and raymath.h) , the main file (main.c) and the failure.c file containing all the important 4 functions.
The simulation runs 2 approaches, a yaw free controller which we discussed earlier and a Altitude PID controller which just tries to maintain the altitude and velocity.
The 4 main functions are -

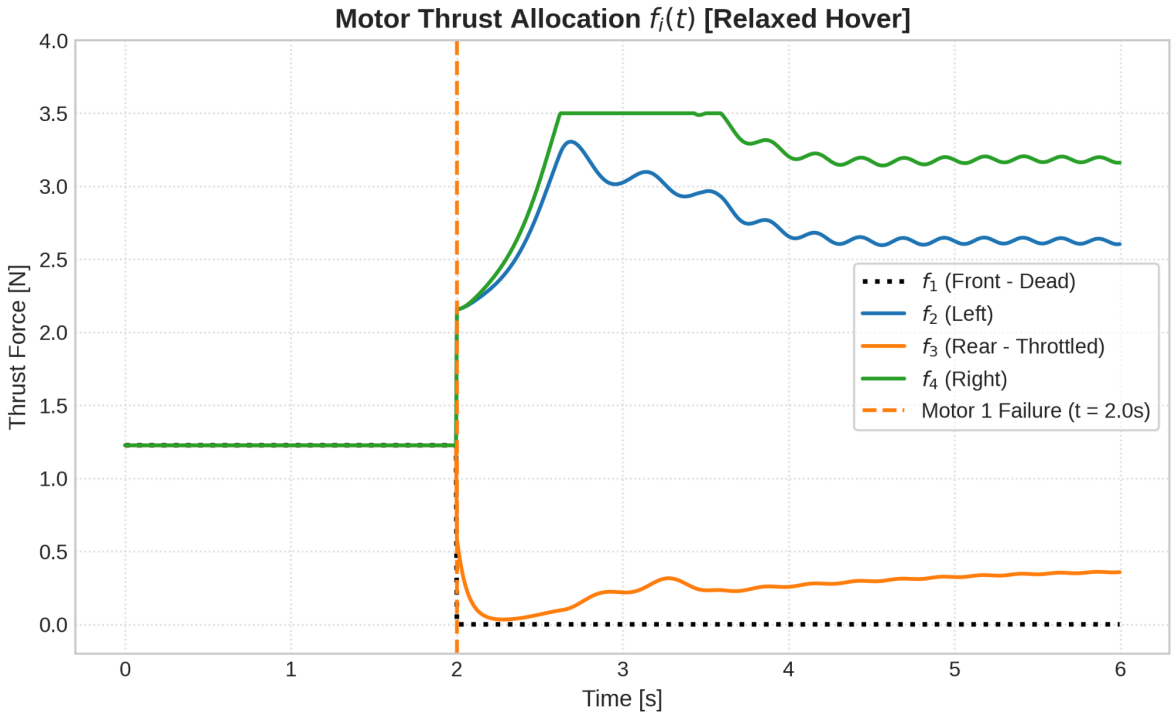
- drone_init() - Which initialises the drone parameters
- drone_step() - Which calculates the change in angular velocity using the equations we derived and updates all the drone parameters using simple euler integration for a time step of 0.001 second.
- control_stock_pid() - which updates the command forces for all the rotors just using simple altitude pid.
- control_relax_hover() - which updates the command forces for the rotors using the relaxed over method which minimizes f_3 and lets yaw spin run free.
- run_benchmark() - which is a wrapper around all the prev mentioned functions which loops 1000 times a second and calculates all the data required and updates that into csv files for both the approaches

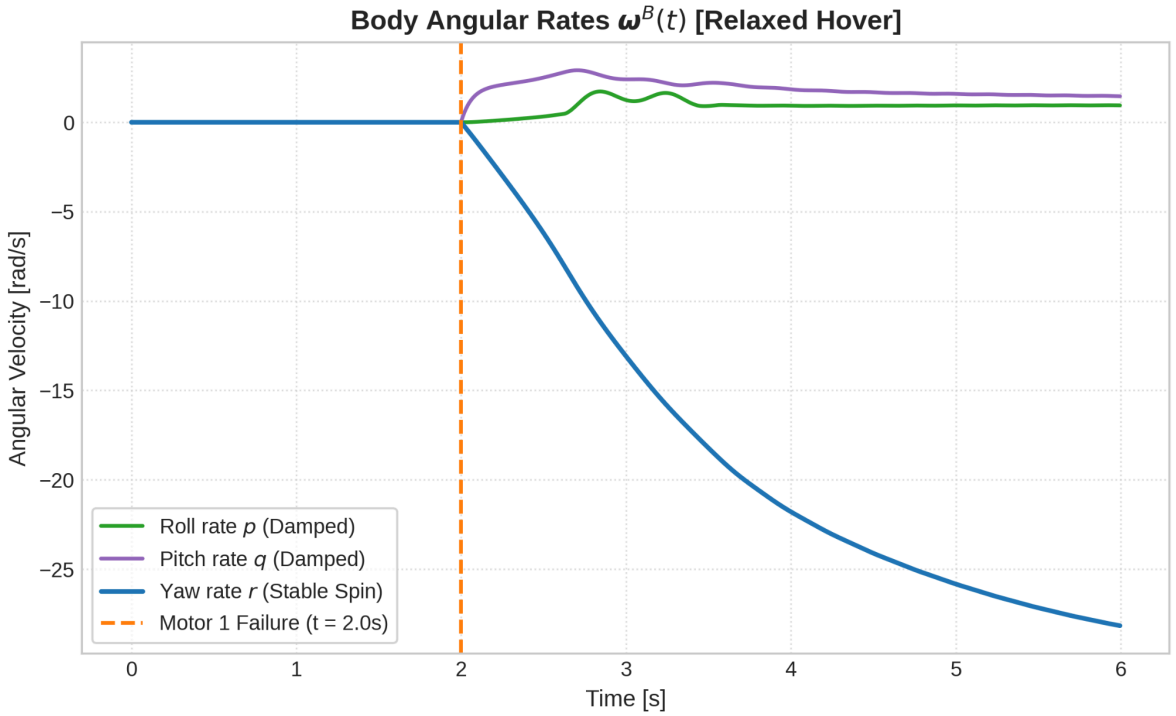
Then I have a python script to plot the csv data using matplotlib and a makefile for compilation.

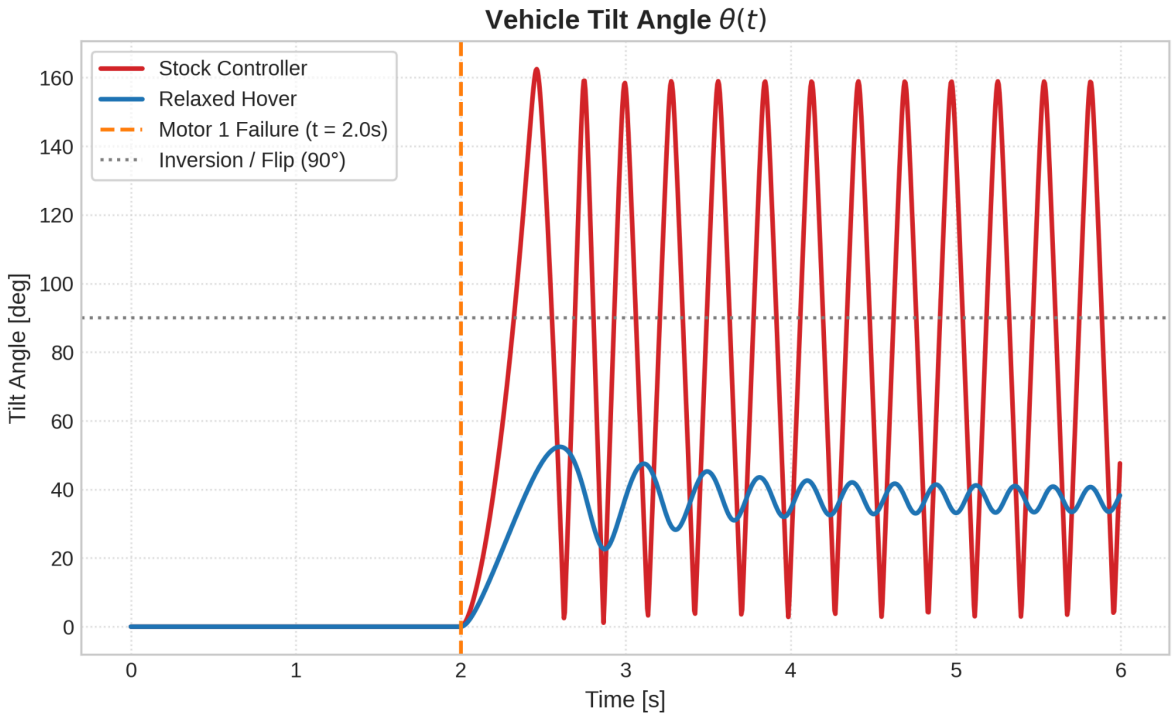
Github link - https://github.com/IceCube1312/motor_failure_thingi
These are some samples of the plotted graphs-

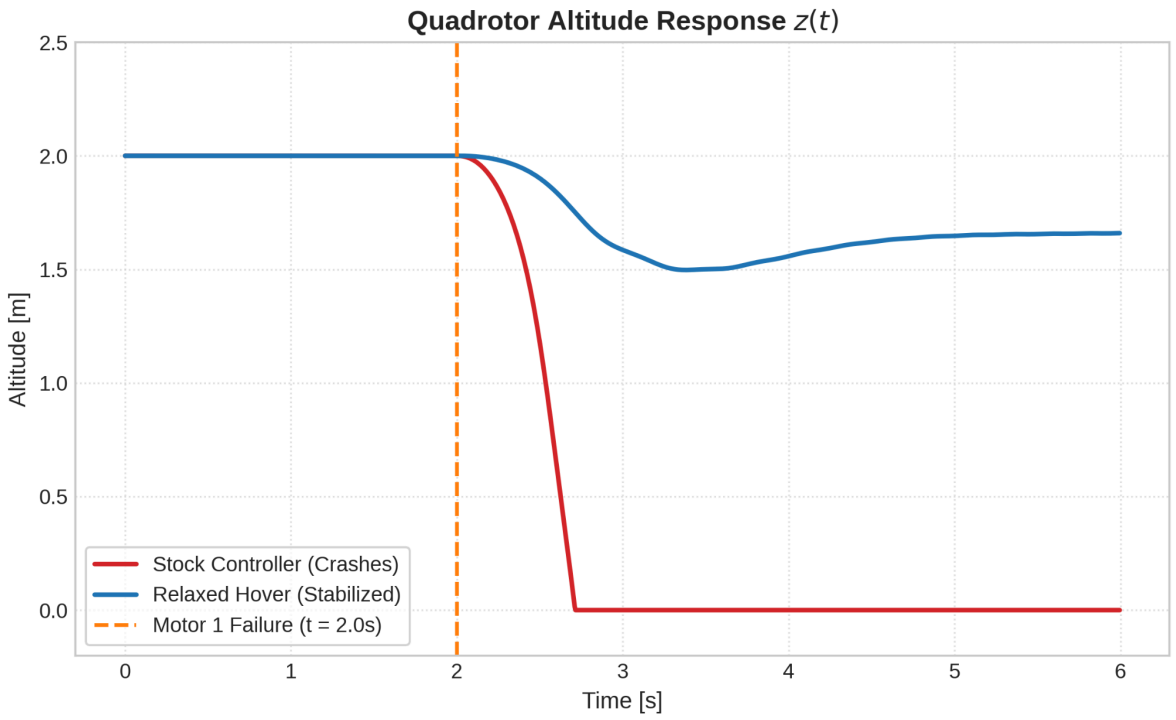
KPI Metric	Stock PID controller	Relaxed Hover
Altitude loss	2m	0.5m
Maximum Tilt	162.6*	52.4*
Horizontal Drift (at t=4s)	0.73m	11.86m

Recovery Time	Failed (crashed)	1.32 sec
Steady State yaw spin	-4.6 rad/s	-27.1 rad/s









Task 1a:

I went over four public solutions from the Inter -IIT 13.0 archives to understand how previous teams approached fault detection and recovery-

Institute Name	Fault Injection method	Detection Method and Signal	Controller type	Tier	Hardware Tested?
IIT Kanpur	Built a custom module inside px4 firmware that flagged a motor as dead over the internal uORB signal system	Explored a classical residual observer and a ML- based classifier	Used a NMPC which repeatedly solves an optimization solver to get the best motor commands	Achieved hover, landing and return in Gazebo simulation	No

IIT KGP	Automated fault-injection in PX4 Gazebo environment.	Apparent force divergence which looks for divergence in the apparent force in the IMU itself rather than the change in calculated angular acceleration,	MRFTC - References an ideal drone and uses adaptive feedback to force to drone to be closer to the virtual ideal.	Hover, Controlled landing and return to home	Yes on Pixhawk 6C flight controller .
IIT Gandhinagar	Used ROS 2 to send zero thrust over their communication protocol	LSTM Neural Network	Used direct motor rpm commands	Stable hover and landing	No
Team 67	Python script overriding motor commands	Multi-sensor classifier on IMU and Magnetometer Data	NPMC + reinforcement learning	Stable Hover	No

Which detection signal did each use?

- IIT Kanpur: Angular Acceleration Residual (instantaneous torque imbalance from gyro).
- IIT Kharagpur: Apparent Force Divergence (instantaneous thrust drop from accelerometer).
- IIT Gandhinagar: Deep Learning (LSTM) on IMU time-series data.
- Team 67: Neural Network (MLP) on combined IMU and magnetometer telemetry.

Which controller did each use?

- IIT Kanpur: NMPC (Nonlinear Model Predictive Control).
- IIT Kharagpur: MRFTC with Yaw surrendered (Model Reference control matching a relaxed hover model).
- IIT Gandhinagar: Offboard low-level allocation.
- Team 67: NMPC + Reinforcement Learning (PPO).

Trade-off Insight (Your Voice):

- Physics-based methods (Kanpur & Kharagpur) are the best: they detect failures in under 40 ms, require zero training, and don't get fooled by wind gusts.
- Machine learning methods (Gandhinagar & Team 67) are academically interesting, but suffer from computational delay (> 80 ms) and risk false alarms under real-world turbulence.

Task 1c:

Reviewing the Inter-IIT 13.0 solutions shows several gaps between simulation and real physical flight:

- Only Tested in Stationary Hover: Most teams only tested cutting a motor while hovering still. In real missions failures happen during fast forward flight (10 - 15 m/s), where aerodynamic drag makes the drone flip much faster.

- No Diagnosis of Failure Type: Teams assumed a motor just cuts to 0. In reality, failures vary: a broken propeller vibrates violently and corrupts gyro sensors while a damaged motor might still produce 40% thrust, confusing a simple controller.
- Inaccurate Gazebo Aerodynamics: Default Gazebo uses a basic thrust formula and ignores air resistance on the spinning drone frame. This causes simulated drones to spin at unrealistic speeds compared to real flight.
- Missing Standard .ulg Flight Logs: Many teams saved custom CSV files instead of native PX4 .ulg binary logs, making it harder to verify sensor noise and control latency.
- Cannot Fly Forward While Spinning: The controllers only hover in place. Spinning at 260 RPM blurs cameras and sensors, making forward navigation impossible without advanced cyclic pitch control.
- Hardware Limits (Sensor Saturation & Heat): Standard drone gyroscopes max out at 2000°/s—if the spin exceeds this, attitude tracking fails completely. Also, the two working motors must run at 90% throttle, overheating the electronics within 1–2 minutes.
- Too Computationally Heavy for Real Microcontrollers: Techniques like NMPC and LSTM neural networks require a heavy companion computer (like an Nvidia Jetson). They cannot run directly on low-power Pixhawk microcontrollers (STM32).